NETWORK PACKET SNIFFER

RASHID IQBAL
--(AI RED TEAMER)--

PROJECT 1ST AS CYBER ROLE AT ARCH TECHNOLOGIES

BASIC NETWORK SNIFFER

In this project, I built a Basic Network Sniffer using Python to capture and analyze live network traffic flowing through my system, specifically working on the network interface supported by **Linux using AF_PACKET (such as eth0 or wlan0)**. I used the socket module to create a raw socket that listens to all incoming and outgoing packets in real time from that interface. With the help of the struct module, I unpacked raw binary data into meaningful information such as MAC addresses, IP addresses, ports, and protocol types. I first extracted the Ethernet frame to identify the source, destination, and protocol being used. Then, I processed IPv4 packets to obtain details like version, header length, TTL, and IP addresses. Based on the protocol field, I handled ICMP, TCP, and UDP separately using dedicated functions. For TCP, I extracted ports, sequence numbers, acknowledgment numbers, and control flags to understand connection behavior. For UDP and ICMP, I retrieved their respective header details and payload data. I also formatted the raw packet data into readable hexadecimal output using textwrap for better clarity and structure. Through this project, I gained practical understanding of packet structures, network layers, and how data flows across a Linux network interface in real time.

OBJECTIVE:

The main goal of this project was to:

1) Capture live network traffic using **Python**.
2) Understand the structure of network packets at different layers: Ethernet, IP, TCP / UDP / ICMP.
3) Display packet details in a readable format to study how computers communicate over a network.

KEY CONCEPTS LEARNED:

ETHERNET FRAME:

- Every packet starts with an **Ethernet frame**, which is like the **"envelope"** of the message.
- It contains:

DESTINATION MAC	The device the packet is for
SOURCE MAC	The device sending the packet
PROTOCOL TYPE	Identifies what protocol is carried inside (IPv4, ARP, etc)

- Learning Ethernet frames helped me understand the first layer of network communication.

IPV4 PACKET:

- IPv4 packets are the main structure for sending data over the internet.
- Each IPv4 packet contains:

VERSION & HEADER LENGTH	The device the packet is for
TTL (TIME TO LIVE)	The device sending the packet
PROTOCOL	Identifies what protocol is carried inside (IPv4, ARP, etc)
SOURCE & DESTINATION IP	Shows the sender & receiver addresses

- This taught me how computers identify each other & route information.

TCP SEGMENT:

- TCP is a **connection-based protocol**.
- TCP segments include:

SOURCE & DESTINATION PORTS	Identify the application / service (like HTTP or SSH)
SEQUENCE & ACKNOWLEDGEMENT NUMBER	Manage reliable delivery
FLAGS (SYN, ACK, FIN, etc)	Control connection setup & termination

- By reading TCP flags, I can track how connections start, maintain, and end.

UDP SEGMENT:

- UDP is a **connectionless protocol**.
- UDP packets include:
  - Source & Destination Ports.
  - Length of the data.
- Unlike TCP, UDP does not guarantee delivery.
  It's faster & used for things like **DNS queries** or **streaming**.
- Understanding UDP helped me learn the difference between reliable & fast protocols.

ICMP PACKET:

- ICMP is used for network diagnostics, like **ping**.
- ICMP packet have:

TYPE & CODE	Define the purpose of the message (echo request, echo reply, etc)
CHECKSUM	Ensures data integrity

- ICMP packets helped me understand **network troubleshooting messages**.

RAW DATA & HEXADECIMAL REPRESENTATION:

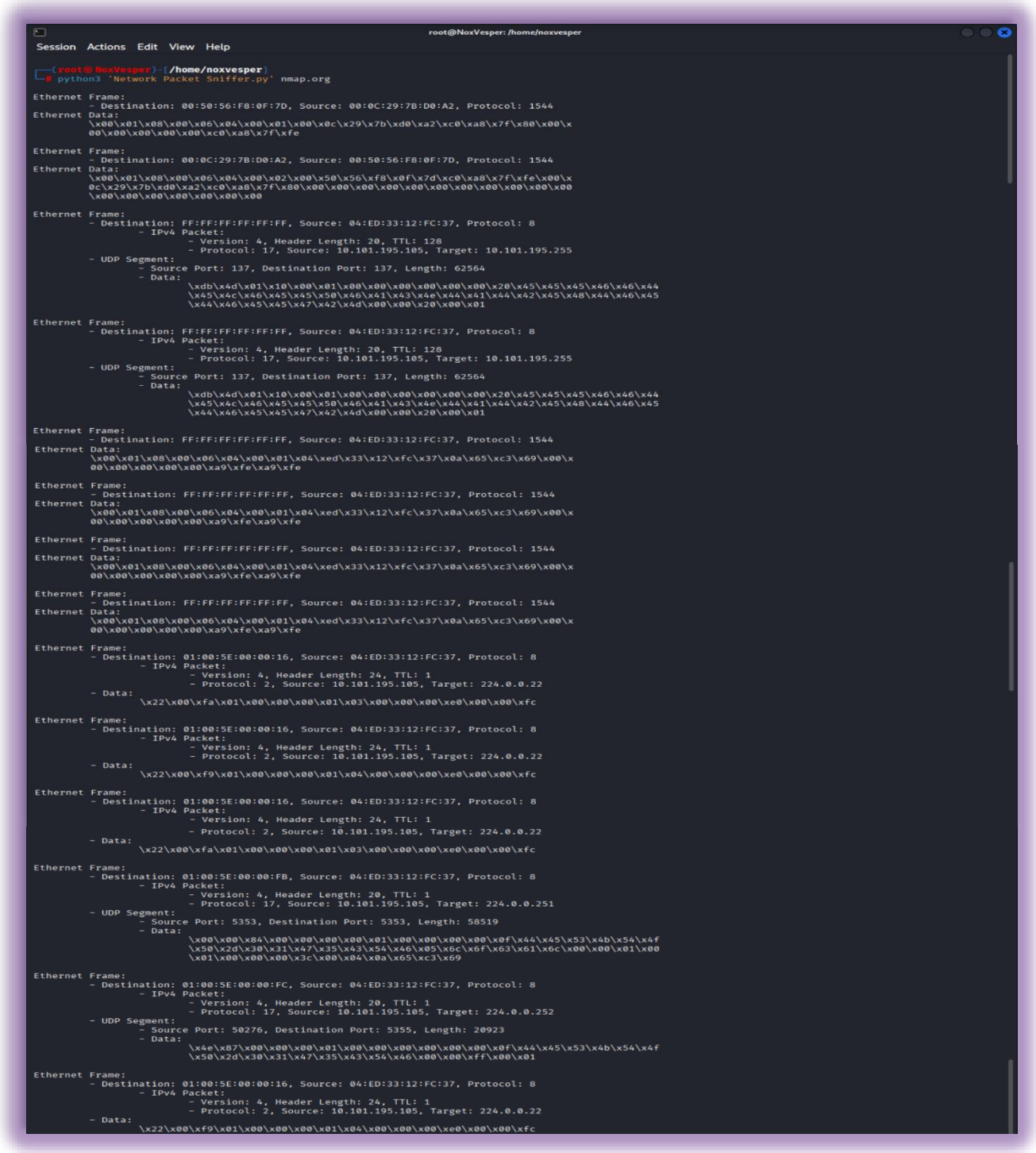
- Some packets contain **unknown or unsupported protocols**, which are displayed as raw hexadecimal data.
- This shows the actual bytes being sent & helps me analyze unusual or custom protocols.

PYTHON IMPLEMENTATION:

- I used **Python raw sockets** to capture packets.
- Functions were created to:
  - Parse Ethernet frames.
  - Parse IPv4 headers.
  - Extract TCP, UDP, and ICMP data.
  - Format the output neatly for readability.
- This made me realize how Python can interact directly with network interfaces to inspect traffic.

PRACTICAL USE:

- This sniffer helps me:
  - Learn network communication in real time.
  - Understand packet structures.
  - Detect unusual or suspicious activity.
  - Debug & analyze network traffic.
- Essentially, it's like seeing the invisible data **"conversations"** between devices.



CONCLUSION:

Exploring the hidden world of network traffic taught me valuable skills & insights…

I

Built a Packet Sniffer to monitor & analyze network traffic.
Gained **hands-on experience** with network layers, protocols, and packet structures.
Learned to capture, parse, and interpret live network data using **Python**.
Developed a **deeper understanding** of how devices communicate over a network.
Strengthened my knowledge in **Networking**, **Cybersecurity**, and **Ethical Hacking**.
Prepared myself for real-world tasks in **network analysis & security testing**.